

# Jenkins, CI/CD & GitHub Integration

- CI/CD: Continuous Integration/Continuous Delivery, Continuous Deployment  
CI: **Continuous Integration** is a DevOps practice where developers frequently integrate their code changes into a shared repository. Each integration automatically triggers build, automated tests and code quality.  
CD: **Continuous Delivery** extends CI by automating the testing and preparation of code for release. This process ensures that the application can be released to production at any time with minimal risk.  
CD: **Continuous Deployment** represents the final stage of CI/CD pipeline. It takes automation a step further by automatically deploying code to production without manual approval.
- Jenkins Architecture:  
It's an open-source automation server for CI/CD, streamlining software building, testing and deployment boost efficiency and reduce errors.  
**Jenkins Master/Master Node** : This is the central controller of the Jenkins environment. It's responsible for managing jobs, scheduling, pipelines, credentials, plugins.  
**Jenkins Agents/Slave Node**: This handles the build and test execution as assigned by master, helping distribute the load and enable parallel processing.
- Freestyle Jobs:  
A freestyle job is Jenkins job configured through the Jenkins web interface.  
Use-Case: Running custom scripts(e.g. Shell, python)  
Deploying a basic application  
Running a system maintenance tasks.
- Declarative Pipeline:  
A declarative pipeline defines Jenkins CI/CD workflow using structured pipeline syntax. The implemented pipeline contains three stages: Checkout, Build and Test. The pipeline configuration is stored as code in Jenkinsfile and versioned-controlled with the application.
- Jenkinsfile  
The declarative pipeline is stored as a code in Jenkinsfile. The key fields this pipeline contains are:
  1. Pipeline: Defines entire pipeline
  2. Agent: It specifies where the pipeline will run
  3. Environment: Sets the environment variables required in the pipeline.
  4. Stages: Contains individual stage blocks for different steps
  5. Steps: Defines the commands, instructions required to execute within stage.

6. Post: Defines action which will be performed after the pipeline execution. In both the cases, if the pipeline fails this post block will be executed and even after the success this block will be executed.

Example:

```
Pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Build') {
            steps {
                sh '''
                    cd Week5/task
                    python3 -m venv myenv
                    myenv/bin/pip install -r requirements.txt
                '''
            }
        }
        stage('Test') {
            steps {
                sh '''
                    cd Week5/task
                    myenv/bin/pytest
                '''
            }
        }
    }
}
```

- GitHub Webhook:

A GitHub webhook allows a GitHub to send HTTP notification to Jenkins when an event such as repository push occurs. For this implementation, the Jenkins freestyle job was configured with 'GitHub hook trigger for GITScm Polling' option. Because Jenkins was initially running on localhost, ngrok was used as a temporary HTTP tunnel so GitHub could reach the local Jenkins instance. The webhook was configured for push events with JSON payload and SSL verification enabled.

- Jenkins Credentials:  
Jenkins Credentials provide secure storage for sensitive information such as GitHub access token, SSH keys, Usernames/Passwords, API credentials. Credentials should be stored in the Jenkins Credentials rather than hard-coded into a Jenkinsfile or shell script.

Credential -> Jenkins Credentials store -> Jenkins Job/Pipeline

- Pipeline stages:  
The stages I implemented while working on this task are: Checkout, Build, Test.
  1. Checkout:  
Purpose: This retrieves the source code from the GitHub.
  2. Build:  
Purpose: The prepare the application for the execution. Creates python virtual environment and install the required dependencies.
  3. Test:  
Purpose: This stage validate the application by running the test. In this, pytest is used to test the application.

The pipeline successfully executed two automated pytest tests. The final result was 2 PASSED and FINISHED: SUCCESS

Name: Purva Wankhede

Batch: B1

Date: 12 Aug 2026